

Git Cheat Sheet — by Azmi

This cheat sheet is organized by workflow stages, from initial setup to emergency rollbacks. The goal is to keep your repository clean, collaborate safely, and ensure you never lose or overwrite critical assignment code.

1. Setup & Initialization

Commands to start a new project or configure your environment.

Command	Description
<code>git config --global user.name "Name"</code>	Set the name attached to your commits.
<code>git config --global user.email "email"</code>	Set the email attached to your commits.
<code>git init</code>	Initialize a local Git repository in the current directory.
<code>git clone <url></code>	Clone an existing remote repository to your local machine.

2. The Daily Bread & Butter

The standard loop for saving and tracking your work.

Command	Description
<code>git status</code>	Show modified files, staged files, and untracked files. (Run this constantly)
<code>git add <file></code>	Stage a specific file for the next commit.
<code>git add .</code>	Stage all modified and new files in the current directory.
<code>git commit -m "Message"</code>	Commit your staged changes with a descriptive message.
<code>git commit --amend -m "Msg"</code>	Replace the last commit with a new one (fixes a bad commit message or forgotten files).

3. Branching & Context Switching

Never work directly on `main`. Always branch out for features or assignments to avoid breaking a working state.

Command	Description
<code>git branch</code>	List all local branches. The active branch has a <code>*</code> .
<code>git branch <branch-name></code>	Create a new branch.
<code>git switch <branch-name></code>	Switch to an existing branch. (Modern alternative to <code>git checkout</code>).
<code>git switch -c <branch-name></code>	Create a new branch and switch to it immediately.
<code>git branch -d <branch-name></code>	Delete a local branch safely (prevents deletion if unmerged).

4. Syncing & Team Collaboration

Commands for pushing your work and pulling your teammates' work.

Command	Description
<code>git fetch</code>	Download history and changes from the remote repo, but do not merge them yet.
<code>git pull</code>	Fetch changes from the remote and instantly merge them into your active branch.
<code>git push origin <branch></code>	Push your local branch commits to the remote repository.
<code>git push -u origin <branch></code>	Push a new branch and set the remote tracking link for future <code>git pull</code> / <code>push</code> commands.

5. Merging & Integration

Combining work. This is where merge conflicts happen, so proceed with caution.

Command	Description
<code>git merge <branch-name></code>	Merge the specified branch into your currently active branch.

Command	Description
<code>git rebase <base-branch></code>	Reapply your current branch's commits on top of another branch (keeps a linear history).
<code>git merge --abort</code>	Emergency Button: If you get overwhelmed by merge conflicts, abort the merge and return to the pre-merge state.

6. Stashing (Quick Saves)

Use these when you need to switch branches but aren't ready to commit your half-finished work.

Command	Description
<code>git stash</code>	Temporarily shelve (stash) all modified tracked files.
<code>git stash pop</code>	Restore the most recently stashed files and remove them from the stash list.
<code>git stash list</code>	View all stashed changes.
<code>git stash drop</code>	Discard the most recent stash permanently.

7. History & Inspection

Figuring out who did what, and when.

Command	Description
<code>git log</code>	Show the commit history for the active branch.
<code>git log --oneline --graph</code>	Show a condensed, visual graph of the commit history and branches.
<code>git diff</code>	Show the exact line-by-line code changes in unstaged files.
<code>git diff --staged</code>	Show the code changes in files you have already <code>git add</code> ed.

8. Undoing Mistakes (Saving the Assignment)

How to fix things when you've messed up your code or your repository state.

Command	Description
<code>git restore <file></code>	Discard uncommitted local changes to a file, returning it to the last committed state.
<code>git restore --staged <file></code>	Unstage a file (undo a <code>git add</code>), but keep your code modifications.
<code>git revert <commit-hash></code>	Create a <i>new</i> commit that undoes the changes of a specific past commit (safest way to undo pushed history).
<code>git reset --soft HEAD~1</code>	Undo the last commit, but keep all your code changes staged and ready.
<code>git reset --hard HEAD~1</code>	Warning: Undo the last commit AND delete all code changes you made in it. Use with extreme caution.